Enterprise App & Service Principal Governance (App Registrations, API Permissions)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Enterprise App & Service Principal Governance (Microsoft Entra) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Enterprise App & Service Principal Governance 5

The Entra application model . . . 5

Delegated vs. application permissions . . . 5

Consent — who approves the permissions . . . 5

The over-permission problem . . . 5

Secret and certificate hygiene . . . 6

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 6

2. Requirements Guide — Enterprise App & Service Principal Governance 7

2.1 Entra ID licensing . . . 7

2.2 Administrative roles . . . 7

2.3 Consent policy decision . . . 7

2.4 App inventory approach . . . 7

2.5 Governance scope group (pilot) . . . 8

2.6 Microsoft Graph PowerShell SDK . . . 8

2.7 Break-glass accounts . . . 8

Checklist . . . 8

3. Implementation Guide — Enterprise App & Service Principal Governance 9

Step 1 — Create the governance scope and reviewer groups . . . 9

Step 2 — Inventory every enterprise app and service principal . . . 9

Step 3 — Review API permissions and consents for over-privilege . . . 9

Step 4 — Configure the user-consent policy . . . 10

Step 5 — Enable the admin-consent workflow . . . 10

Step 6 — Remediate risky grants (staged) . . . 10

Step 7 — Manage secret and certificate expiry . . . 10

Step 8 — Validate on the pilot . . . 11

Step 9 — Expand tenant-wide . . . 11

5. Testing Guide — Enterprise App & Service Principal Governance 11

Test 1 — Inventory is complete . . . 12
Test 2 — Over-permission report is accurate . . . 12
Test 3 — User-consent policy is set . . . 12
Test 4 — Consent-phishing is blocked . . . 12
Test 5 — Admin-consent workflow works . . . 12
Test 6 — Risky grant remediation (preview then apply) . . . 12
Test 7 — Credential expiry surfaced . . . 12
Pass/fail gate . . . 12
6. Troubleshooting Guide — Enterprise App & Service Principal Governance 13
A. Discovery or report returns nothing / errors . . . 13
B. Unowned or unknown apps . . . 13
C. A legitimate app broke after remediation . . . 13
D. User-consent policy not taking effect . . . 14
E. Admin-consent requests not reaching reviewers . . . 14
F. Consent-phishing still succeeding . . . 14
G. Diagnostics quick reference . . . 14
H. When to escalate . . . 14
7. Best Practices — Enterprise App & Service Principal Governance 15
8. Deliverables Summary — Enterprise App & Service Principal Governance 15
Documentation . . . 15
PowerShell scripts . . . 16
What the buyer receives . . . 16
Outcome . . . 16
Appendix A — PowerShell Scripts 18

Executive Summary — Enterprise App & Service Principal Governance (Microsoft Entra)

White-label · Entra ID Solutions

Every SaaS integration and internal app you connect leaves behind three things in your tenant: an enterprise application, a service principal, and a set of API permissions. Over time they pile up — apps nobody owns, dangerous consented permissions, and app secrets or certificates quietly expiring or lingering forever. Attackers love them: an over-permissioned or consent-phished app is a quiet, persistent foothold that survives password resets and MFA.

This is a complete, deploy-it-today package for governing enterprise apps and service principals in Microsoft Entra, written for an administrator with no prior app-governance experience. It gives you a repeatable way to inventory them, review consent and API permissions for over-privilege, control user consent, and manage secret and certificate hygiene.

What you get

- A plain-language Solution Overview of the Entra app model — app registrations vs. enterprise

apps and service principals, delegated vs. application permissions, admin and user consent, the over-permission and consent-phishing risk, and the secret/certificate lifecycle.

- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, click-by-click Entra steps in the required order.
- Five PowerShell scripts for app and service-principal discovery, risky-consent and expiring-secret

validation, an over-permission and expiring-secret report, safe remediation of risky grants, and a governance-scope group builder.

- Testing and Troubleshooting guides for a confident, staged rollout.

What this delivers

A governed application estate — every enterprise app and service principal inventoried, API permissions and consents reviewed for over-privilege, a user-consent policy that stops consent-phishing, an admin-consent workflow so users can request access safely, risky grants remediated, and app secrets and certificates on a managed lifecycle — proven on a pilot scope before you touch the whole tenant.

The standard deployment model

Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every enterprise app and service principal inventoried and governed — over-permissioned grants and risky consents remediated, user consent controlled, and app secrets and certs on a managed lifecycle — closing a favorite attacker foothold, validated on a pilot before you scale.

1. Solution Overview — Enterprise App & Service Principal Governance

The Entra application model

When an app integrates with Microsoft Entra, two related objects appear. An `application` (the app registration) is the global definition of the app — its identity, its declared permissions, and its credentials — and it lives in the tenant that authored it. A `servicePrincipal` (the enterprise application) is the local instance of that app in your tenant: it is what you actually assign users to, consent permissions to, and see under Enterprise applications. A multi-tenant SaaS vendor authors one app registration and every customer that signs in gets a service principal. Governance is mostly about service principals — the running instances in your tenant — plus the app registrations you own.

Delegated vs. application permissions

Permissions come in two flavors, and the difference is the heart of the risk.

- Delegated permissions act on behalf of a signed-in user and are capped by that user's own rights.

Example: `Mail.Read` delegated lets the app read only the signed-in user's mail.

- Application permissions (app roles) act as the app itself, with no user present, across the whole

tenant. Example: `Mail.Read` as an application permission lets the app read every mailbox in the tenant. These are the dangerous ones — a compromised app with tenant-wide application permissions is a tenant-wide breach.

In the directory, granted delegated permissions appear as `oauth2PermissionGrants`, and granted application permissions appear as `appRoleAssignments` on the service principal.

Consent — who approves the permissions

- User consent: a user clicks Accept on an app's consent prompt and grants delegated permissions

for themselves. Convenient, but it is exactly how consent-phishing works — an attacker sends a link to a malicious app that requests broad delegated scopes (read all your mail, files, send mail as you), and one careless click hands over a durable OAuth token that survives password changes and MFA.

- Admin consent: an administrator grants permissions for the whole organization — required for

application permissions and for high-privilege delegated scopes. The healthy pattern is to restrict user consent to low-risk permissions and route everything else through an admin-consent workflow, so users can request an app and an admin reviews it before it is trusted.

The over-permission problem

Apps accumulate more permission than they use. A vendor's app requests broad scopes to cover every possible feature; an internal script is granted `Directory.ReadWrite.All` because it was quicker than scoping it down. Every unused permission is blast radius: if that app or its secret is stolen, the attacker inherits everything it was granted. Governance means finding these over-permissioned apps — especially

those with tenant-wide application permissions — and cutting them back to what they need.

Secret and certificate hygiene

An app authenticates with a client secret (`passwordCredentials`) or a certificate (`keyCredentials`). Two failure modes matter: a credential that expires unexpectedly breaks the integration (an outage), and a credential that never expires — or has a very long life, or is one of many piled on the same app — is a standing risk that outlives the people who created it. Governance tracks expiry so nothing breaks by surprise, and prunes stale or excessive credentials so a leaked secret has a short, bounded life.

What you can control

- Inventory: every enterprise app and service principal, its owners, its app-role assignments, its OAuth2 delegated grants, and its credentials.
- User-consent policy: block user consent, or allow it only for verified publishers and low-risk permissions; enable an admin-consent workflow so users can request review.
- Over-permission review: identify apps holding high-privilege application permissions or broad delegated grants, and remediate the risky ones.
- Credential lifecycle: track secret and certificate expiry, and remove stale or surplus credentials.

Benefits

- Security — the number-one payoff: risky consents revoked, over-privilege cut, and consent-phishing blocked at the policy level before a user can be tricked.
- Visibility — a real inventory of what apps exist, who owns them, and what they can do.
- Continuity — no more integrations breaking because a secret expired unnoticed.
- Repeatability — a governance pattern you run on a schedule, not a one-time cleanup.

Limitations (be honest with buyers)

- Revoking a permission or removing a credential can break a legitimate app — every change is staged, validated on a pilot scope, and reversible, and the remediation script defaults to a preview (`-WhatIf`).
- The admin-consent workflow and some risk features need Entra ID P1 or P2 (see requirements).
- This governs Entra app permissions and consent; it does not replace a full third-party SaaS security review of what each vendor does with the access.

The standard deployment model

Inventory apps and service principals -> review permissions and consent for over-privilege -> configure the user-consent policy and admin-consent workflow on a pilot scope -> validate -> remediate risky grants and manage credential expiry -> expand tenant-wide. repeatable pattern: Assess -> Configure -> Pilot -> Validate -> Expand.

2. Requirements Guide — Enterprise App & Service Principal Governance

Verify every requirement before configuring. Each entry: Description / Why / Verify / Implement-if-missing. Category-wide prerequisites are in `global-requirements-entra-id-solutions.md`.

2.1 Entra ID licensing

Description: Microsoft Entra ID P1 (P2 if you also want risk-based and identity-protection signals). Why: The admin-consent workflow and some app-governance and risk features need P1+; the core app and service-principal inventory and consent settings work with any tenant, but the review workflow that makes this repeatable needs P1. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2. Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

2.2 Administrative roles

Description: An operator role that can read and manage apps and consent. `Application Administrator` (or `Cloud Application Administrator`) for day-to-day app governance; `Privileged Role Administrator` or `Global Administrator` only for the settings that change tenant-wide consent policy. Why: Reviewing and remediating app permissions, consent grants, and credentials requires an app-management role; changing the user-consent policy is a tenant-wide, high-privilege action. Verify: Entra -> `Roles and administrators`; confirm `Application Administrator` (and a `Global Administrator` available for the consent-policy change). Implement: Assign `Application Administrator` (or `Cloud Application Administrator`); reserve `Global Administrator` for the one-time consent-policy change.

2.3 Consent policy decision

Description: A documented decision on how much users may consent to on their own — block user consent, or allow it only for verified publishers and low-risk delegated permissions — and whether to enable the admin-consent workflow. Why: It is the single most consequential control for stopping consent-phishing. The default in many tenants is far too permissive. Verify: Entra -> `Identity -> Applications -> Enterprise applications -> Consent and permissions -> User consent settings`; confirm the current setting is a deliberate choice, not the default. Implement: Default to "Allow user consent for apps from verified publishers, for selected permissions" (low-risk delegated scopes only); enable the admin-consent workflow so users can request apps for review. Decide in Implementation Step 4.

2.4 App inventory approach

Description: A repeatable way to enumerate every enterprise app and service principal with its app-role assignments, delegated grants, owners, and credentials. Why: You cannot govern what you have not inventoried; the tenant almost always holds more apps than anyone expects. Verify: Run `APPGOV-Discovery.ps1`; confirm it returns service principals with their app-role assignments, OAuth2 grants, and credential expiry. Implement: Install the Microsoft Graph PowerShell SDK (see 2.6) and run `APPGOV-Discovery.ps1`.

2.5 Governance scope group (pilot)

Description: A pilot scope for the consent-policy and workflow rollout — a security group of representative users named `App Governance Consent Scope - Pilot - Pilot`, plus a reviewer group `App Governance Consent Reviewers - Tenant - Steady` for the admin-consent workflow. Why: Every change is validated on a pilot before the tenant. The user-consent policy is tenant-wide, so the pilot gate is the admin-consent workflow rollout and the remediation of a first batch of apps — not a per-user policy split. Verify: The groups exist with representative members; run `APPGOV-Set-ScopeTargeting.ps1` to preview. Implement: Create the groups with `APPGOV-Set-ScopeTargeting.ps1` (see Implementation Step 1).

2.6 Microsoft Graph PowerShell SDK

Description: The Microsoft Graph PowerShell SDK on the operator workstation, with consent to the read scopes the scripts use. Why: Discovery, validation, and reporting read service principals, applications, app-role assignments, OAuth2 grants, credentials, and the authorization policy through Microsoft Graph. Verify: `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; a test `Connect-MgGraph` succeeds. Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`; the scripts request the least privilege they need (`Application.Read.All`, `Directory.Read.All`, `Policy.Read.All`, and — only for remediation — `Application.ReadWrite.All` / `DelegatedPermissionGrant.ReadWrite.All`).

2.7 Break-glass accounts

Description: Two cloud-only emergency access accounts excluded from broad policies. Why: A consent-policy or app-permission change could interfere with admin tooling; break-glass accounts restore access. Verify: Entra -> `Users`; two cloud-only `@onmicrosoft.com` admins exist (see global requirement G4). Implement: Create them, exclude from broad policies, store credentials securely, and monitor sign-ins.

Checklist

- Entra ID P1/P2 licensing assigned
- `Application Administrator` role assigned; `Global Administrator` available for consent-policy change
- User-consent policy decision documented (verified-publisher, low-risk scopes) + admin-consent workflow decision
- App inventory produced — `APPGOV-Discovery.ps1`

- Governance scope + reviewer groups created — `APPGOV-Set-ScopeTargeting.ps1`
- Microsoft Graph PowerShell SDK installed and consented
- Two break-glass accounts created and excluded from broad policies

3. Implementation Guide — Enterprise App & Service Principal Governance

Follow these steps in order. Most steps are in the Microsoft Entra admin center (`https://entra.microsoft.com`); the inventory and remediation steps use the included scripts.

> *Model: inventory first, review permissions and consent for over-privilege, set the user-consent policy and admin-consent workflow, then remediate a first batch of risky apps and manage credential expiry on a pilot scope before expanding tenant-wide. Every change is staged and reversible.*

Step 1 — Create the governance scope and reviewer groups

1. Entra -> `Identity -> Groups -> All groups -> New group` -> Security ->

`App Governance Consent Scope - Pilot - Pilot` (a few representative users). Create.

1. Create `App Governance Consent Reviewers - Tenant - Steady` — the admins who will review

admin-consent requests. Add the reviewers.

1. Or run `APPGOV-Set-ScopeTargeting.ps1` to create and populate the groups and preview membership.

Step 2 — Inventory every enterprise app and service principal

1. Entra -> `Identity -> Applications -> Enterprise applications -> All applications` to see the

service principals in the tenant.

1. Run `APPGOV-Discovery.ps1`. It exports, per service principal: the app-role assignments

(application permissions), the OAuth2 delegated grants, the owners, and the client-secret and certificate expiry. Save the output — it is your baseline.

Step 3 — Review API permissions and consents for over-privilege

1. For a suspect app: Entra -> `Enterprise applications ->` open the app `-> Permissions`. Review the

admin-consented permissions; note any tenant-wide application permissions (for example `Mail.Read`, `Directory.ReadWrite.All`, `Application.ReadWrite.All`).

1. Run `APPGOV-Report.ps1` to rank apps by risk — those holding high-privilege application

permissions, broad delegated grants, many credentials, or an unknown/absent owner rise to the top.

1. Build a shortlist of over-permissioned and unowned apps to remediate in Step 6.

Step 4 — Configure the user-consent policy

1. Entra -> `Identity -> Applications -> Enterprise applications -> Consent and permissions ->

User consent settings`.

1. Set `User consent for applications` to `Allow user consent for apps from verified publishers, for

selected permissions (`low-risk delegated scopes only`). `Do not leave it on` Allow user consent for all apps`.

1. Under `Consent and permissions -> Permission classifications`, confirm only low-risk delegated

permissions (for example `openid`, `profile`, `User.Read`, `offline_access`) are classified as low.

1. Save. This is the tenant-wide control that stops consent-phishing.

Step 5 — Enable the admin-consent workflow

1. Entra -> `Identity -> Applications -> Enterprise applications -> Admin consent settings`.

2. Set `Users can request admin consent to apps they are unable to consent to` to `Yes`.

3. Add `App Governance Consent Reviewers - Tenant - Steady` as the reviewers who receive and act on

requests; set the notification and request-expiry options. Save.

Step 6 — Remediate risky grants (staged)

1. Preview first: run `APPGOV-Remediation.ps1` with its default `-WhatIf`. It reports the risky

OAuth2 delegated grants and app-role assignments it would revoke — nothing is changed.

1. Review the preview against the Step 3 shortlist with the app owners; confirm each app can lose the

grant without breaking a needed function.

1. Re-run with `-WhatIf:$false` scoped to the confirmed apps to revoke the risky grants. To revoke a

single app in the portal instead: `Enterprise applications ->` open the app `-> Permissions ->` remove the permission, or `Properties -> Delete` to remove an unused app entirely.

Step 7 — Manage secret and certificate expiry

1. Run `APPGOV-Report.ps1` (expiring-secret section) to list credentials expiring soon and

credentials with no expiry or an excessive lifetime.

1. For an app you own: `Entra -> App registrations ->` open the app `-> Certificates & secrets`;

rotate the secret or certificate before expiry and remove stale or surplus credentials.

1. Prefer a certificate over a long-lived client secret for app-only authentication.

Step 8 — Validate on the pilot

1. Run `APPGOV-Validation.ps1` -> confirms the user-consent policy is set to verified-publisher / low-risk, the admin-consent workflow is enabled with reviewers, and no pilot-scope app still holds a revoked risky grant.

1. Have a pilot user attempt to consent to a high-risk app -> the attempt should be blocked and route to an admin-consent request. Complete the Testing Guide.

Step 9 — Expand tenant-wide

1. With the policy validated, work the full Step 3 shortlist in batches — remediate the remaining over-permissioned and unowned apps with `APPGOV-Remediation.ps1` (preview, confirm, apply).

1. Put credential expiry on a schedule: re-run `APPGOV-Report.ps1` monthly and rotate ahead of expiry.

1. Re-run `APPGOV-Discovery.ps1` and `APPGOV-Report.ps1` on a recurring cadence so new apps and grants are caught as they appear.

Order of operations

```
Scope + reviewer groups -> inventory -> permission/consent review ->
user-consent policy -> admin-consent workflow -> remediate risky grants ->
credential expiry -> validate -> expand
```

Rollback

• User-consent policy: revert `User consent settings` to the previous option in `Consent and permissions` (change is immediate and reversible).

• Admin-consent workflow: set `Users can request admin consent...` back to `No`.

• Revoked grant: re-add the permission on the app's `Permissions` blade (or re-grant admin consent); because remediation is confirmed per app from a `-WhatIf` preview, only reviewed apps are ever changed.

5. Testing Guide — Enterprise App & Service Principal Governance

Run after the pilot configuration applies (Implementation Step 8) and on the recurring governance cadence.

Test 1 — Inventory is complete

Do: Run `APPGOV-Discovery.ps1`. Expected: Every enterprise app and service principal is listed with its app-role assignments, OAuth2 delegated grants, owners, and credential expiry — a usable baseline.

Test 2 — Over-permission report is accurate

Do: Run `APPGOV-Report.ps1` and spot-check one flagged app in the portal (`Enterprise applications ->` app `-> Permissions`). Expected: The high-privilege application permissions and broad delegated grants the report flags match what the portal shows.

Test 3 — User-consent policy is set

Do: Run `APPGOV-Validation.ps1`, or check `Consent and permissions -> User consent settings`. Expected: User consent is `Allow user consent for apps from verified publishers, for selected permissions` — not `Allow user consent for all apps`.

Test 4 — Consent-phishing is blocked

Do: As a pilot user, attempt to consent to a high-risk / unverified test app requesting broad delegated scopes. Expected: The self-consent is blocked and the user is offered an admin-consent request instead.

Test 5 — Admin-consent workflow works

Do: The pilot user submits the request from Test 4. Expected: A reviewer in `App Governance Consent Reviewers - Tenant - Steady` receives the request and can approve or deny it.

Test 6 — Risky grant remediation (preview then apply)

Do: Run `APPGOV-Remediation.ps1` (default `-WhatIf`) against a confirmed over-permissioned app, then re-run with `-WhatIf:$false`. Expected: The preview lists exactly the grant it will revoke; after applying, `APPGOV-Validation.ps1` confirms the grant is gone and the app no longer holds it.

Test 7 — Credential expiry surfaced

Do: Run `APPGOV-Report.ps1` (expiring-secret section). Expected: Secrets and certificates expiring soon, and any with no expiry or an excessive lifetime, are listed with their app and days-to-expiry.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1-5 pass; a first over-permissioned app is remediated (Test 6) and validated
Expand pass	Tests 1-3, 7 pass tenant-wide; the risky-app shortlist is worked and credential expiry is on a schedule

6. Troubleshooting Guide — Enterprise App & Service Principal Governance

Use `APPGOV-Discovery.ps1` for inventory, `APPGOV-Report.ps1` for over-permission and expiring-secret status, `APPGOV-Validation.ps1` for policy state, and `APPGOV-Remediation.ps1` (preview first) for safe fixes.

A. Discovery or report returns nothing / errors

Symptom	Cause	Remediation
`Connect-MgGraph` fails or scopes are missing	SDK not installed, or consent to read scopes not granted	`Install-Module Microsoft.Graph -Scope CurrentUser`; re-run and accept the `Application.Read.All`, `Directory.Read.All`, `Policy.Read.All` scopes
Empty results	Signed in to the wrong tenant, or an app-read role missing	Confirm the tenant; ensure the operator holds `Application Administrator` / `Cloud Application Administrator`
Owners show as blank	The app genuinely has no owner	Expected — flag it; assign an owner (see B)

B. Unowned or unknown apps

- An app with no owner and unclear purpose is the highest-priority review item. Check its `passwordCredentials` / `keyCredentials` and recent sign-ins, identify the vendor from the app name or publisher, assign an owner, and if it is unused remove it (`Enterprise applications ->` app `-> Properties -> Delete`).

C. A legitimate app broke after remediation

Symptom	Cause	Remediation
Integration fails after a grant was revoked	The revoked permission was actually in use	Re-add the permission on the app's `Permissions` blade (or re-grant admin consent); revert is immediate. Always run `APPGOV-Remediation.ps1` in `-WhatIf` first and confirm with the owner

Symptom	Cause	Remediation
App-only job fails at a certain date	The client secret or certificate expired	Rotate the credential in App registrations -> Certificates & secrets; track future expiry with APPGOV-Report.ps1

D. User-consent policy not taking effect

- After changing User consent settings, confirm you saved and that you edited the tenant-wide

setting under Consent and permissions, not a single app. Run APPGOV-Validation.ps1 to read back the effective authorizationPolicy consent setting; the change is tenant-wide and near-immediate.

E. Admin-consent requests not reaching reviewers

- Confirm Admin consent settings -> Users can request admin consent... is Yes, the reviewer group

App Governance Consent Reviewers - Tenant - Steady is set as reviewers, and notifications are enabled. Reviewers must hold a role able to grant tenant-wide consent.

F. Consent-phishing still succeeding

- If a user still consented to a broad app, a high-risk delegated permission is likely classified as

low-risk. Review Consent and permissions -> Permission classifications and remove anything but genuinely low-risk scopes (for example keep only openid, profile, User.Read, offline_access). Revoke the bad grant with APPGOV-Remediation.ps1.

G. Diagnostics quick reference

```
Connect-MgGraph -Scopes 'Application.Read.All','Directory.Read.All','Policy.Read.All' -NoWelcome
# effective user-consent policy
(Invoke-MgGraphRequest -Method GET -Uri '/v1.0/policies/authorizationPolicy').defaultUserRolePermissions
# an app's granted application permissions and delegated grants
Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId <id>
Get-MgOauth2PermissionGrant -Filter "clientId eq '<id>'"
# an app's credentials
(Get-MgServicePrincipal -ServicePrincipalId <id>).PasswordCredentials
(Get-MgServicePrincipal -ServicePrincipalId <id>).KeyCredentials
```

H. When to escalate

If an app cannot be attributed to any owner or vendor, holds tenant-wide application permissions, and shows unexpected sign-in activity, treat it as a potential compromise: revoke its grants and credentials, disable the service principal (AccountEnabled false), and engage your incident-response process.

7. Best Practices — Enterprise App & Service Principal Governance

• Restrict user consent — the single most important control. Allow user consent only for verified publishers and low-risk delegated permissions; never leave it on `Allow user consent for all apps`. This is what stops consent-phishing before a user can be tricked.

• Turn on the admin-consent workflow. Blocking user consent without a request path just drives shadow IT; let users request an app and have a named reviewer group approve or deny it.

• Treat application permissions as tenant-wide breach potential. Scrutinize every app holding application permissions (especially `*.ReadWrite.All`, `Directory.*`, `Mail.*`, full-mailbox or full-directory scopes); grant them only to apps that truly need them, and only by admin consent.

• Enforce least privilege. Cut every app back to the permissions it actually uses; unused permission is pure blast radius if the app or its secret is stolen.

• Every app needs an owner. Unowned apps are how sprawl becomes risk — assign an owner to each app so someone can answer for it and rotate its credentials.

• Prefer certificates over long-lived secrets. Use a certificate for app-only authentication, keep secret lifetimes short, and never let a credential sit with no expiry.

• Track expiry proactively. Run the report on a schedule and rotate before expiry so integrations never break by surprise; prune stale and surplus credentials so a leaked secret has a short life.

• Stage and preview every removal. Revoking a permission or deleting an app can break a legitimate integration — always run remediation in `-WhatIf` preview first, confirm with the owner, then apply to the reviewed apps only.

• Make it recurring, not one-time. New apps and grants appear constantly; re-run discovery and the report on a cadence so governance is a standing process.

• Document the estate. Record each app's owner, purpose, permissions, and credential-rotation date for audits and repeatability.

8. Deliverables Summary — Enterprise App & Service Principal Governance

Documentation

Document	Purpose
Executive Summary	Business case and what is in the box

Document	Purpose
1. Solution Overview	The Entra app model, delegated vs. application permissions, consent and consent-phishing, over-permission, secret/certificate lifecycle
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Entra ID Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps (inventory, permission/consent review, user-consent policy, admin-consent workflow, remediate, credential expiry, validate, expand)
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`APPGOV-Discovery.ps1`	Inventories enterprise apps and service principals with app-role assignments, OAuth2 delegated grants, owners, and credential expiry via Microsoft Graph (read-only)
`APPGOV-Validation.ps1`	Confirms the user-consent policy, admin-consent workflow, and that remediated apps no longer hold risky grants (read-only)
`APPGOV-Report.ps1`	Over-permission and expiring-secret report — ranks apps by permission risk and lists credentials expiring or with no expiry (read-only)
`APPGOV-Remediation.ps1`	Safely revokes risky OAuth2 grants and app-role assignments; `-WhatIf` by default
`APPGOV-Set-ScopeTargeting.ps1`	Builds and previews the governance consent-scope and reviewer groups

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable process: Assess -> Configure -> Pilot -> Validate -> Expand.

Outcome

Every enterprise app and service principal inventoried and governed — over-permissioned grants and risky consents remediated, user consent controlled, and app secrets and certs on a managed lifecycle — closing a favorite attacker foothold, proven on a pilot before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPGOV-Discovery.ps1

Purpose: Read-only inventory of every service principal (enterprise application) in the tenant. Per app it reports: the application (app-role) permissions granted via appRoleAssignments, the delegated permissions granted via oauth2PermissionGrants, the owners, and the client-secret (passwordCredentials) and certificate (keyCredentials) expiry. This is your governance baseline. HOW TO RUN Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\APPGOV-Discovery.ps1 Optional: -CsvPath .\appgov-inventory.csv to export a flat CSV. Consents to Application.Read.All, Directory.Read.All, Policy.Read.All (read-only). EXPECTED RESULTS A per-service-principal listing of application permissions, delegated grants, owners, and credential expiry. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([string]$CsvPath)

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once powershell -ExecutionPolicy Bypass -File .\APPGOV-Discovery.ps1 Optional: -CsvPath .\appgov-inventory.csv to export a flat CSV. Consents to Application.Read.All, Directory.Read.All, Policy.Read.All (read-only). EXPECTED RESULTS A per-service-principal listing of application permissions, delegated grants, owners, and credential expiry. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([string]$CsvPath)

APPGOV-Remediation.ps1

Purpose: Removes risky permissions from a named, confirmed over-permissioned app: its delegated oauth2PermissionGrants and/or its application-permission appRoleAssignments. It is intended to be run against apps you have already reviewed from APPGOV-Report.ps1 and confirmed with the owner. Defaults to -WhatIf, so a plain run only PREVIEWS what it would revoke and changes nothing. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPGOV-Remediation.ps1 -AppId <appId> Apply after review: powershell -ExecutionPolicy Bypass -File .\APPGOV-Remediation.ps1 -AppId <appId> -WhatIf:$false Scope: -RevokeDelegated / -RevokeAppRoles (default: both). Consents to Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All. EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run APPGOV-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES Workplace Marketplace — Entra ID Solutions. Write action — preview (-WhatIf) by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\APPGOV-Remediation.ps1 -AppId <appId> Apply after review: powershell -ExecutionPolicy Bypass -File .\APPGOV-Remediation.ps1 -AppId <appId> -WhatIf:$false Scope: -RevokeDelegated / -RevokeAppRoles (default: both). Consents to Application.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All, Directory.Read.All. EXPECTED RESULTS In preview: the exact delegated grants and app-role assignments that WOULD be revoked. When applied: those grants/assignments are removed; re-run APPGOV-Validation.ps1 -CheckAppId to confirm. Reversible by re-granting the permission in the portal. .NOTES Workplace Marketplace — Entra ID Solutions. Write action — preview (-WhatIf) by default. #>

[CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$AppId, [switch]$RevokeDelegated, [switch]$RevokeAppRoles)

APPGOV-Report.ps1

Purpose: Read-only risk report over every service principal. It (1) ranks apps by permission risk — those holding high-privilege application permissions (appRoleAssignments to *.ReadWrite.All, Directory.*, Mail.*, full-mailbox/full-directory scopes), broad delegated grants (oauth2PermissionGrants), no owner, or many credentials — and (2) lists client secrets (passwordCredentials) and certificates (keyCredentials) that are expired, expiring within a window, or have no expiry / an excessive lifetime. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPGOV-Report.ps1 Optional: -ExpiryWindowDays 30 (default) to set the "expiring soon" window. Consents to Application.Read.All, Directory.Read.All (read-only). EXPECTED RESULTS A ranked over-permission table and an expiring-secret list with app, credential type, and days-to-expiry. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([int]$ExpiryWindowDays = 30)

How to run: powershell -ExecutionPolicy Bypass -File .\APPGOV-Report.ps1 Optional: -ExpiryWindowDays 30 (default) to set the "expiring soon" window. Consents to Application.Read.All, Directory.Read.All (read-only). EXPECTED RESULTS A ranked over-permission table and an expiring-secret list with app, credential type, and days-to-expiry. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([int]$ExpiryWindowDays = 30)

APPGOV-Set-ScopeTargeting.ps1

Purpose: Creates (or previews) the two security groups this solution targets: the pilot consent scope "App Governance Consent Scope - Pilot - Pilot" and the admin-consent reviewer group "App Governance Consent Reviewers - Tenant - Steady". Use the reviewer group in the admin-consent workflow (Implementation Step 5) and the pilot scope to stage the rollout. By default it PREVIEWS (no changes); pass -Create to create groups that do not yet exist. HOW TO RUN Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\APPGOV-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\APPGOV-Set-ScopeTargeting.ps1 -Create Consents to Group.ReadWrite.All, Directory.Read.All (read-only in preview mode). EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create)

How to run: Preview which groups exist / would be created (read-only): powershell -ExecutionPolicy Bypass -File .\APPGOV-Set-ScopeTargeting.ps1 Create the groups if missing: powershell -ExecutionPolicy Bypass -File .\APPGOV-Set-ScopeTargeting.ps1 -Create Consents to Group.ReadWrite.All, Directory.Read.All (read-only in preview mode). EXPECTED RESULTS In preview: whether each named group already exists and its member count. With -Create: the missing groups are created (empty) and their object IDs returned so you can add members. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only unless -Create is supplied. #> [CmdletBinding()] param([switch]$Create)

APPGOV-Validation.ps1

Purpose: Read-only check that the governance controls are in place: (1) the tenant user-consent policy restricts user consent (verified publishers / low-risk permissions, not "all apps"), read from the

authorizationPolicy; (2) the admin-consent workflow is enabled with reviewers; and (3) an optional named app no longer holds a revoked delegated grant or app-role assignment. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPGOV-Validation.ps1 Optional: -CheckAppId <appId> to verify a remediated app no longer holds risky grants. Consents to Policy.Read.All, Application.Read.All, Directory.Read.All (read-only). EXPECTED RESULTS PASS/WARN lines for the user-consent policy and admin-consent workflow, and — if -CheckAppId is given — the current app-role assignments and delegated grants for that app. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([string]$CheckAppId)

How to run: powershell -ExecutionPolicy Bypass -File .\APPGOV-Validation.ps1 Optional: -CheckAppId <appId> to verify a remediated app no longer holds risky grants. Consents to Policy.Read.All, Application.Read.All, Directory.Read.All (read-only). EXPECTED RESULTS PASS/WARN lines for the user-consent policy and admin-consent workflow, and — if -CheckAppId is given — the current app-role assignments and delegated grants for that app. No changes are made. .NOTES Workplace Marketplace — Entra ID Solutions. Read-only. #> [CmdletBinding()] param([string]$CheckAppId)